

Wazuh VirusTotal Integration

Malware Detection and Threat Intelligence Using Wazuh SIEM and VirusTotal Integration

TECHNICAL IMPLEMENTATION REPORT

Prepared By

Jithin Raj E

Aspiring SOC Analyst | Cybersecurity Enthusiast

Cybersecurity Portfolio Project

This project demonstrates the implementation of an open-source Security Information and Event Management (SIEM) solution integrated with VirusTotal threat intelligence to detect, enrich, and investigate potentially malicious files in a monitored environment..

CORE DOMAINS

- SIEM Monitoring
- Threat Intelligence Integration
- File Integrity Monitoring (FIM)
 - Malware Detection
- Alert Triage and Analysis
- Incident Investigation

may 2026

Table of Contents

Table of Contents	2
1. Abstract	4
2. Introduction.....	4
2.1 Background.....	4
2.2 Motivation	5
2.3 Problem Statement	5
3. Objectives.....	6
3.1 Primary Objectives.....	6
3.2 Secondary Objectives	6
4. Scope	6
4.1 In Scope.....	6
4.2 Out of Scope	7
5. Architecture and Working of the Solution.....	7
5.1 High-Level Architecture	7
5.2 Component Descriptions.....	7
5.3 Detection Workflow	8
6. Lab Environment Setup	9
6.1 Virtualisation Platform	9
6.2 Virtual Machines	9
6.3 Network Configuration	9
7. Wazuh Installation and Configuration	10
7.1 Wazuh Manager Deployment.....	10
7.2 Verifying Wazuh Manager Service Status	10
7.3 Agent Installation and Registration.....	12
8. VirusTotal API Integration	13
8.1 Obtaining the VirusTotal API Key	13
8.2 Configuring ossec.conf	13
8.3 Custom Detection Rules	14
9. Attack Demonstration and Testing	16
9.1 Test Methodology.....	16
9.2 Test Procedure.....	16
9.3 Generated Alerts and Dashboard Evidence	16
10. Dashboard Visualisation.....	17
10.1 Wazuh Dashboard Overview	17
10.2 Endpoints Management	18
10.3 Kali Agent Detail View	18

10.4 Threat Hunting — Pre-Test Dashboard State.....	19
10.5 Threat Hunting — Post-Test Dashboard State	20
10.6 Threat Hunting Events — VirusTotal Alerts	21
11. Advantages and Limitations	23
11.1 Advantages	23
11.2 Limitations	23
12. Conclusion.....	25
13. References	27

1. Abstract

The proliferation of malware and sophisticated cyber threats in modern computing environments necessitates robust, automated mechanisms for real-time detection and alerting. This project presents the design, implementation, and evaluation of an integrated malware detection system that leverages Wazuh, a leading open-source Security Information and Event Management (SIEM) platform, in conjunction with the VirusTotal threat intelligence API. The system was deployed on a virtualized laboratory environment consisting of a Wazuh Manager running on Ubuntu 22.04 LTS and a monitored endpoint running Kali GNU/Linux 2025.1.

The integration operates by enabling the Wazuh File Integrity Monitoring (FIM) module to detect file creation and modification events on the agent host. Upon detection, the VirusTotal integration module automatically submits the file hash to the VirusTotal cloud service for reputation analysis. If the file is identified as malicious by VirusTotal, a custom rule (Rule ID 100500, Level 15) triggers a high-priority alert that is visible in the Wazuh Dashboard. The system was tested using the industry-standard EICAR test file, which successfully generated critical-level threat alerts. This project demonstrates a practical, cost-effective approach to real-time endpoint protection that is directly applicable to Security Operations Centre (SOC) workflows. The findings confirm that open-source SIEM platforms can be effectively extended with third-party threat intelligence to deliver enterprise-grade malware detection capabilities.

2. Introduction

2.1 Background

The modern threat landscape has evolved dramatically, with malicious actors deploying increasingly sophisticated malware to compromise endpoints, steal sensitive data, and disrupt organisational operations. Traditional signature-based antivirus solutions have proven insufficient against zero-day exploits, polymorphic malware, and fileless attacks. Consequently, Security Operations Centres (SOCs) have adopted multi-layered defence

strategies that incorporate behavioural analysis, threat intelligence feeds, and automated alerting systems.

Security Information and Event Management (SIEM) platforms occupy a central role in this defence architecture, aggregating log data from disparate sources, correlating events, and generating actionable alerts for security analysts. Among open-source SIEM solutions, Wazuh has emerged as a widely adopted platform owing to its comprehensive feature set, active community support, and extensible integration capabilities.

2.2 Motivation

While Wazuh provides powerful built-in detection capabilities through its FIM, rootcheck, and log analysis engines, its effectiveness can be substantially amplified by integrating external threat intelligence services. VirusTotal, operated by Google, aggregates scan results from over 70 antivirus engines and URL scanners, providing one of the most comprehensive file reputation databases available. By combining Wazuh's real-time file monitoring with VirusTotal's crowdsourced threat intelligence, it becomes possible to detect malicious files with a high degree of confidence, even in the absence of local antivirus software.

This project was undertaken with the specific goal of gaining hands-on experience with SOC analyst workflows, understanding the architecture of SIEM platforms, and demonstrating the practical application of threat intelligence APIs in an automated detection pipeline. The knowledge gained from this project is directly relevant to roles in cybersecurity operations, incident response, and threat analysis.

2.3 Problem Statement

Organisations frequently lack visibility into malicious files that are introduced onto endpoint systems, particularly when those files have not yet been flagged by locally installed security software. There is a need for a system that can automatically detect new or modified files, query a trusted threat intelligence service, and generate prioritised alerts for SOC analysts without requiring manual intervention. This project addresses that need by implementing an automated malware detection pipeline using Wazuh and the VirusTotal API.

3. Objectives

The primary and secondary objectives of this project are enumerated below:

3.1 Primary Objectives

- To install and configure the Wazuh SIEM platform, comprising the Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard, within a virtualised laboratory environment.
- To deploy and register a Wazuh agent on a Kali GNU/Linux 2025.1 endpoint and verify bidirectional communication with the Wazuh Manager.
- To configure the Wazuh File Integrity Monitoring (FIM) module to monitor designated directories on the agent host for file creation, modification, and deletion events.
- To integrate the VirusTotal API with the Wazuh Manager by configuring the integration stanza in the `ossec.conf` configuration file with a valid API key.
- To author custom detection rules in the Wazuh `local_rules.xml` file that trigger high-priority alerts upon receipt of a positive VirusTotal threat detection.
- To validate the complete detection pipeline using the EICAR standard antivirus test file and confirm that alerts are generated and displayed correctly in the Wazuh Dashboard.

3.2 Secondary Objectives

- To document the architecture, configuration steps, and technical challenges encountered during the project, producing a reference for future implementations.
- To develop familiarity with SOC analyst workflows, including the interpretation of alert severity levels, rule logic, and threat intelligence enrichment.
- To demonstrate the extensibility of open-source SIEM platforms for advanced threat detection scenarios.

4. Scope

The scope of this project encompasses the following areas:

4.1 In Scope

- Installation and configuration of Wazuh v4.14.4 Manager, Indexer, and Dashboard on a virtualised Ubuntu 22.04 LTS instance.
- Deployment of a Wazuh agent (v4.14.4) on a Kali GNU/Linux 2025.1 virtual machine.

- Configuration of the VirusTotal integration module within Wazuh, limited to the syscheck (FIM) alert group.
- Creation of custom Wazuh detection rules for VirusTotal-positive findings.
- Testing the detection pipeline using the EICAR test file as a controlled simulation of malicious file introduction.
- Visualisation and analysis of generated alerts within the Wazuh Dashboard, including the Threat Hunting and Security Events modules.

4.2 Out of Scope

- Active threat remediation or automated incident response actions (e.g., file quarantine or agent isolation) are not implemented in this iteration.
- Integration with additional threat intelligence feeds beyond VirusTotal.
- Production deployment on physical hardware or enterprise network infrastructure.
- Analysis of network-layer threats; the project focuses exclusively on endpoint filebased detection.

5. Architecture and Working of the Solution

5.1 High-Level Architecture

The solution comprises three principal components: the Wazuh Agent, the Wazuh Manager (with integrated VirusTotal integration module), and the Wazuh Dashboard. These components interact in a coordinated pipeline to achieve automated malware detection and alert generation.

The architecture follows a client-server model in which the Wazuh Agent continuously monitors the endpoint filesystem and forwards relevant events to the Wazuh Manager over an encrypted channel. The Manager performs rule-based correlation, queries the VirusTotal API for file reputation, and generates alerts that are indexed and presented in the Dashboard.

5.2 Component Descriptions

Wazuh Agent: The Wazuh Agent is a lightweight service installed on the monitored endpoint (Kali Linux). It collects log data, monitors file system changes using the FIM module (syscheck), and transmits event data to the Wazuh Manager. The FIM module computes cryptographic hashes (MD5, SHA1, SHA256) of monitored files and detects additions, modifications, and deletions.

Wazuh Manager: The Wazuh Manager is the central processing node. It receives events from agents, applies detection rules using the `wazuh-analysisd` daemon, and invokes the `wazuh-integratord` daemon to communicate with external APIs. When a `syscheck` event matches the VirusTotal integration configuration, the `integratord` submits the file hash to the VirusTotal API and processes the response.

VirusTotal API: The VirusTotal API v3 accepts file hash queries and returns a JSON response indicating whether the submitted hash is known to be malicious, the number of antivirus engines that detected it, and additional metadata. This response is parsed by the Wazuh integration module to generate enriched alerts.

Wazuh Dashboard: The Wazuh Dashboard, built on OpenSearch Dashboards, provides a web-based graphical interface for visualising security events, alert timelines, agent status, and compliance metrics. Analysts use this interface to investigate alerts generated by the detection pipeline.

5.3 Detection Workflow

The end-to-end detection workflow operates as follows:

1. A file is created or modified in a directory monitored by the Wazuh FIM module on the Kali Linux agent.
2. The `syscheck` module computes the file's cryptographic hash and generates a FIM alert (Rule ID 554: File added to the system or Rule ID 550: Integrity checksum changed).
3. The FIM alert is transmitted to the Wazuh Manager and processed by `wazuhanalysisd`.
4. The `wazuh-integratord` detects that the alert belongs to the `syscheck` group, which is configured for VirusTotal integration.
5. The `integratord` extracts the file hash from the alert and submits an HTTP GET request to the VirusTotal API endpoint (`https://www.virustotal.com/api/v3/files/{hash}`).
6. The VirusTotal API returns a JSON response containing the analysis results from its database of antivirus engines.
7. If the file is identified as malicious, `wazuh-analysisd` processes the response and triggers custom Rule ID 100500 (Level 15: VirusTotal detected a malicious file — High Priority Alert).
8. The alert is indexed in the Wazuh Indexer (OpenSearch) and becomes immediately visible in the Wazuh Dashboard under Threat Hunting > Events.

6. Lab Environment Setup

6.1 Virtualisation Platform

The entire project was implemented within a virtualised environment hosted on Oracle VM VirtualBox. This approach ensured a controlled, isolated network that could be reset and reconfigured without impacting the host system. The use of virtualisation also facilitated the creation of a realistic multi-host network topology at no additional hardware cost.

6.2 Virtual Machines

Two virtual machines were provisioned for this project:

Component	Details
Wazuh Manager VM	Wazuh OVA v4.14.4 (Ubuntu 22.04 LTS base)
Agent VM	Kali GNU/Linux 2025.1
Manager IP	10.178.249.126 (NAT/Host-Only)
Agent IP	10.0.2.15
Wazuh Version	v4.14.4
Agent ID	003 (kali)
Network Mode	NAT + Host-Only Adapter

The Wazuh Manager was deployed using the official Wazuh OVA (Open Virtual Appliance) image, which bundles the Wazuh Manager, Wazuh Indexer (OpenSearch), and Wazuh Dashboard into a single, pre-configured virtual appliance. This significantly reduced installation complexity and ensured component compatibility.

6.3 Network Configuration

Both virtual machines were configured with a Host-Only network adapter to enable communication between the agent and the manager within the isolated virtual network. The Wazuh Manager was assigned a static IP address of 10.178.249.126, while the Kali agent obtained the address 10.0.2.15 via DHCP. The Wazuh Dashboard was accessed from the host machine's browser by navigating to <https://10.178.249.126>.

The Wazuh Manager was deployed by importing the official Wazuh OVA v4.14.4 into Oracle VM VirtualBox. Upon first boot, the virtual machine presented the Wazuh login prompt, as illustrated in Figure 1. The administrator logged in using the default credentials (username: wazuh-user) and was presented with the Wazuh Open Source Security Platform banner, confirming successful initialisation of the appliance.



7.2 Verifying Wazuh Manager Service Status

Page 10 of 27

```

[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-manager
■ wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-05-31 14:48:37 UTC; 1min 48s ago
   Process: 2032 ExecStart=/usr/bin/envu /var/ossec/bin/wazuh-control start (code=exited, status=0/
   Tasks: 233 (limit: 4647)
   Memory: 938.9M
   CPU: 1min 12.632s
   CGroup: /system.slice/wazuh-manager.service
           └─2305 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─2306 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─2307 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─2310 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─2313 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─2343 /var/ossec/bin/wazuh-integratord
                       └─2369 /var/ossec/bin/wazuh-authd
                         └─2389 /var/ossec/bin/wazuh-db
                           └─2445 /var/ossec/bin/wazuh-execd
                             └─2467 /var/ossec/bin/wazuh-analysisd
                               └─2482 /var/ossec/bin/wazuh-syscheckd
                                 └─2504 /var/ossec/bin/wazuh-remoted
                                   └─2593 /var/ossec/bin/wazuh-logcollector
                                     └─2607 /var/ossec/bin/wazuh-monitord
                                       └─2631 /var/ossec/bin/wazuh-modulesd

May 31 14:48:33 wazuh-server envu[2032]: wazuh-logcollector: Process 5739 not used by Wazuh, removing...
May 31 14:48:33 wazuh-server envu[2032]: Started wazuh-logcollector...
May 31 14:48:33 wazuh-server envu[2032]: wazuh-monitord: Process 5756 not used by Wazuh, removing...
May 31 14:48:34 wazuh-server envu[2032]: Started wazuh-monitord...
May 31 14:48:34 wazuh-server envu[2032]: wazuh-modulesd: Process 5766 not used by Wazuh, removing...
May 31 14:48:34 wazuh-server envu[2629]: 2026/05/31 14:48:34 wazuh-modulesd:router: INFO: Loaded rou
May 31 14:48:34 wazuh-server envu[2629]: 2026/05/31 14:48:34 wazuh-modulesd:content_manager: INFO: L
May 31 14:48:34 wazuh-server envu[2629]: 2026/05/31 14:48:34 wazuh-modulesd:inventory-harvester: INF
May 31 14:48:35 wazuh-server envu[2032]: Started wazuh-modulesd...
May 31 14:48:37 wazuh-server envu[2032]: Completed.

```

Figure 2: Wazuh Manager Service Status — Active (Running) State Confirmed

Figure 2 presents the output of the `systemctl status` command for the `wazuhmanager.service` unit. The critical indicator is the green "active (running)" status, which confirms that the Wazuh Manager is fully operational. The service was activated on 202605-31 at 14:48:37 UTC. The CGroup section reveals all the constituent daemon processes that form the Wazuh Manager: `wazuh-apid` (API server), `wazuh-integratord` (third-party integrations), `wazuh-authd` (agent authentication), `wazuh-db` (database), `wazuh-execd` (active responses), `wazuh-analysisd` (event analysis and rule matching), `wazuh-syscheckd` (file integrity monitoring), `wazuh-remoted` (agent communication), `wazuh-logcollector` (log collection), `wazuh-monitord` (monitoring), and `wazuh-modulesd` (extended capabilities). The journal log entries at the bottom confirm that all modules started successfully and completed initialisation.

The memory consumption of 938.9 MB and CPU utilisation of 1 minute 12.632 seconds indicate that the Wazuh Manager was actively processing events at the time of the screenshot. The presence of multiple `wazuh-apid` processes reflects the multi-worker configuration of the Wazuh API server, which handles concurrent API requests from the Dashboard and external integrations.

7.3 Agent Installation and Registration

The Wazuh agent was installed on the Kali GNU/Linux 2025.1 virtual machine by downloading and executing the official Wazuh agent installer package. The agent was registered with the Wazuh Manager using the `agent-auth` utility, which establishes a unique agent identity and certificate-based authentication. The registered agent was assigned Agent ID 003 with the hostname "kali".

Agent connectivity and status were verified through the Wazuh Dashboard Endpoints module, as shown in Figure 6 (Section 10). The agent status was confirmed as "active", indicating successful communication with the Manager.

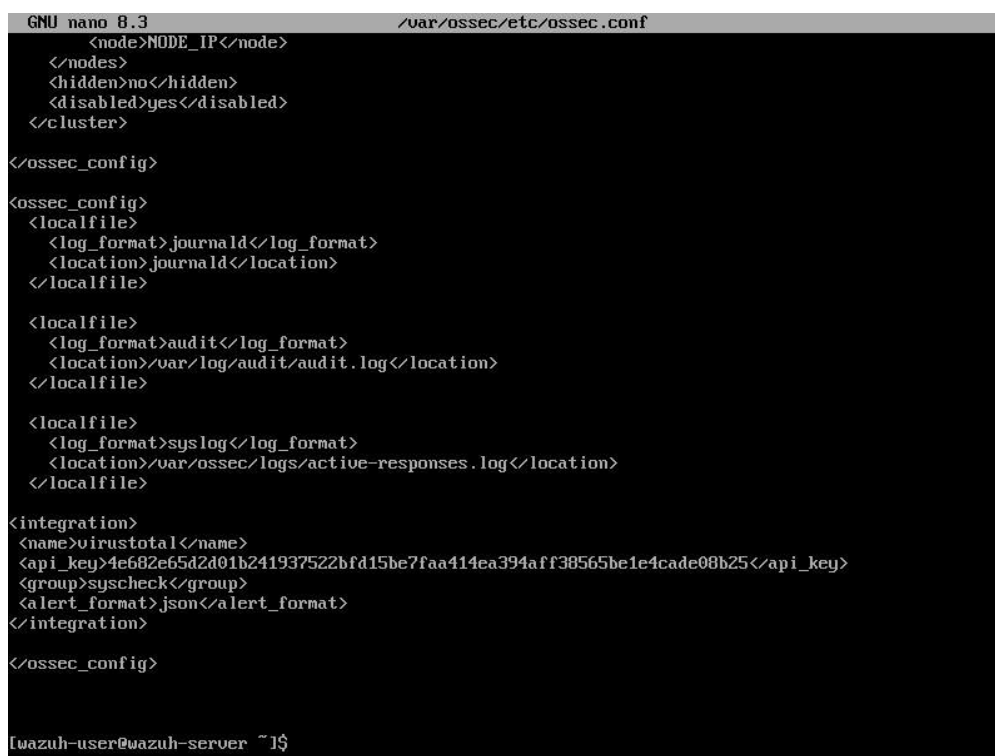
8. VirusTotal API Integration

8.1 Obtaining the VirusTotal API Key

To enable the VirusTotal integration, a free API key was obtained by registering an account at <https://www.virustotal.com>. The free tier of the VirusTotal API permits up to 4 requests per minute and 500 requests per day, which is sufficient for laboratory and smallscale deployment scenarios. The API key is a unique hexadecimal identifier that authenticates API requests submitted by the Wazuh integration module.

8.2 Configuring ossec.conf

The VirusTotal integration was configured by editing the primary Wazuh configuration file located at `/var/ossec/etc/ossec.conf`. The integration stanza was appended to the configuration file, specifying the integration name, API key, monitoring group, and alert format. Figure 3 illustrates the completed configuration.



```
GNU nano 8.3 /var/ossec/etc/ossec.conf
<node>NODE_IP</node>
</nodes>
<hidden>no</hidden>
<disabled>yes</disabled>
</cluster>

</ossec_config>

<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>

  <localfile>
    <log_format>audit</log_format>
    <location>/var/log/audit/audit.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>

  <integration>
    <name>virustotal</name>
    <api_key>4e682e65d2d01b241937522bfd15be7faa414ea394aff38565be1e4cade08b25</api_key>
    <group>syscheck</group>
    <alert_format>json</alert_format>
  </integration>

</ossec_config>

[wazuh-user@wazuh-server ~]$
```

Figure 3: `ossec.conf` — VirusTotal Integration Configuration Stanza

Figure 3 displays the `/var/ossec/etc/ossec.conf` file open in the GNU nano text editor. The configuration file is structured as XML and contains multiple sections governing various

aspects of Wazuh's behaviour. At the bottom of the visible content, the critical `<integration>` block is visible:

- `<name>virustotal</name>` — specifies that the Wazuh integrator should invoke the VirusTotal integration handler.
- `<api_key>...</api_key>` — contains the VirusTotal API key used for authenticating requests.
- `<group>syscheck</group>` — restricts the integration to only process alerts belonging to the syscheck group, i.e., File Integrity Monitoring events. This prevents unnecessary API calls from non-file-related events.
- `<alert_format>json</alert_format>` — instructs the integrator to pass alerts to the integration script in JSON format, enabling structured data extraction.

The preceding `<localfile>` sections configure Wazuh to collect logs from journald, the Linux audit framework, and the Wazuh active-responses log. These sections are part of the default configuration and were left unmodified. The cluster configuration at the top of the file shows a single-node deployment with clustering disabled, which is appropriate for a laboratory environment.

8.3 Custom Detection Rules

Custom detection rules were authored and added to the file `/var/ossec/etc/rules/local_rules.xml`. This file is the designated location for user-defined rules that extend Wazuh's built-in rule set. The rules are evaluated by the `wazuhanalysisd` daemon after all built-in rules have been processed. Figure 4 presents the custom rules created for this project.

```

GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <rule id="100500" level="15">
    <if_group>virustotal</if_group>
    <description>VirusTotal detected a malicious file - High Priority Alert</description>
    <mitre>
      <id>T1204</id>
    </mitre>
  </rule>
</group>

```

Figure 4: local_rules.xml — Custom VirusTotal Detection Rules

Figure 4 shows the local_rules.xml file in the nano editor. Two custom rules are defined within the local,syslog,sshd group:

Rule ID 100001 (Level 5): This rule detects failed SSH authentication attempts from a specific source IP address (1.1.1.1) by matching Snort Incident ID (SID) 5716 and source IP. This rule was included as a reference example and is not directly related to the VirusTotal integration.

Rule ID 100500 (Level 15): This is the primary VirusTotal detection rule. Its components are:

- `<if_group>virustotal</if_group>` — the rule fires when an alert is generated by the VirusTotal integration module.
- `<description>VirusTotal detected a malicious file - High Priority Alert</description>` — the human-readable description displayed in the Wazuh Dashboard.
- `<mitre><id>T1204</id></mitre>` — maps the alert to the MITRE ATT&CK framework technique T1204 (User Execution), indicating that a file requiring user interaction to execute was detected as malicious.

The alert level of 15 is significant: in Wazuh's alert severity scale (0–15), level 15 represents the highest severity. Alerts at this level are classified as "Critical severity" in the Dashboard overview (Rule level 15 or higher). This ensures that VirusTotal-confirmed

malicious file detections receive maximum visibility and priority. The Wazuh Manager service was restarted after saving the rule file to apply the new configuration.

9. Attack Demonstration and Testing

9.1 Test Methodology

The detection pipeline was validated using the EICAR Standard Anti-Virus Test File. The EICAR test file is an industry-standard, inert text file specifically designed for testing antivirus and security software. It is not malware and poses no risk to systems; however, it is universally recognised and flagged as malicious by virtually all antivirus engines, making it an ideal controlled test artefact. The EICAR file's MD5 hash (44d88612fea8a8f36de82e1278abb02f) is well-known to VirusTotal and will return a positive detection result.

9.2 Test Procedure

The EICAR test file was created on the Kali Linux agent within a directory that was configured for monitoring by the Wazuh FIM module (syscheck). The following sequence of events was observed:

9. The EICAR test file was written to the monitored directory using the echo command:
`echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARDANTIVIRUS-TEST-FILE!$H+H*' > /tmp/eicar.com`
10. The Wazuh syscheck module detected the new file and computed its cryptographic hashes.
11. A FIM alert (Rule 554: File added to the system) was generated and forwarded to the Wazuh Manager.
12. The wazuh-integrator detected the syscheck group alert and submitted the file's SHA256 hash to the VirusTotal API.
13. VirusTotal returned a positive detection result, confirming that the file is malicious.
14. The Wazuh Manager processed the VirusTotal response and triggered Rule 100500 (Level 15).
15. Two high-priority alerts appeared in the Wazuh Dashboard within seconds of the file being created.

9.3 Generated Alerts and Dashboard Evidence

The successful execution of the test was confirmed through multiple views in the Wazuh Dashboard, as detailed in the following subsections.

10. Dashboard Visualisation

10.1 Wazuh Dashboard Overview

The Wazuh Dashboard Overview page provides a high-level summary of the security posture of all monitored agents. It displays aggregate alert counts categorised by severity level over a configurable time window.

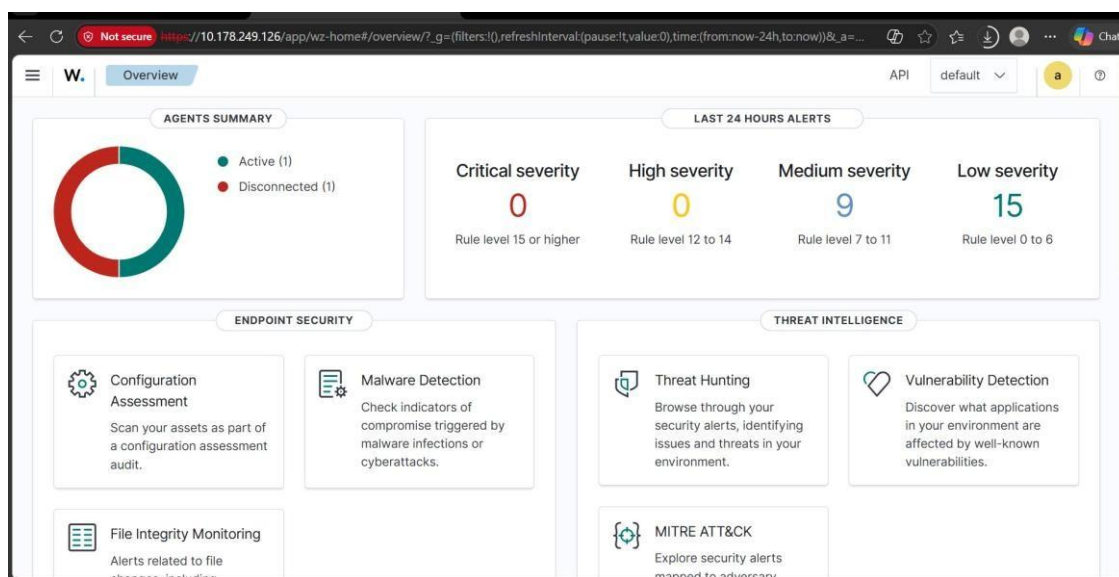


Figure 5: Wazuh Dashboard Overview — Agents Summary and Last 24 Hours Alert Distribution

Figure 5 presents the Wazuh Dashboard Overview accessed at <https://10.178.249.126>. The Agents Summary panel on the left shows one Active agent (the Kali Linux endpoint) and one Disconnected agent (a previously registered Windows agent). The Last 24 Hours Alerts panel on the right shows the distribution of alerts by severity: 0 Critical severity alerts (Rule level 15 or higher), 0 High severity alerts (Rule level 12 to 14), 9 Medium severity alerts (Rule level 7 to 11), and 15 Low severity alerts (Rule level 0 to 6). The absence of Critical severity alerts in this view indicates the screenshot was taken before the EICAR test was executed. The Endpoint Security and Threat Intelligence panels at the bottom provide quick navigation to Wazuh's functional modules including Malware Detection, Threat Hunting, File Integrity Monitoring, Vulnerability Detection, and MITRE ATT&CK mapping.

10.2 Endpoints Management

The Endpoints module provides detailed management and status information for all registered agents.

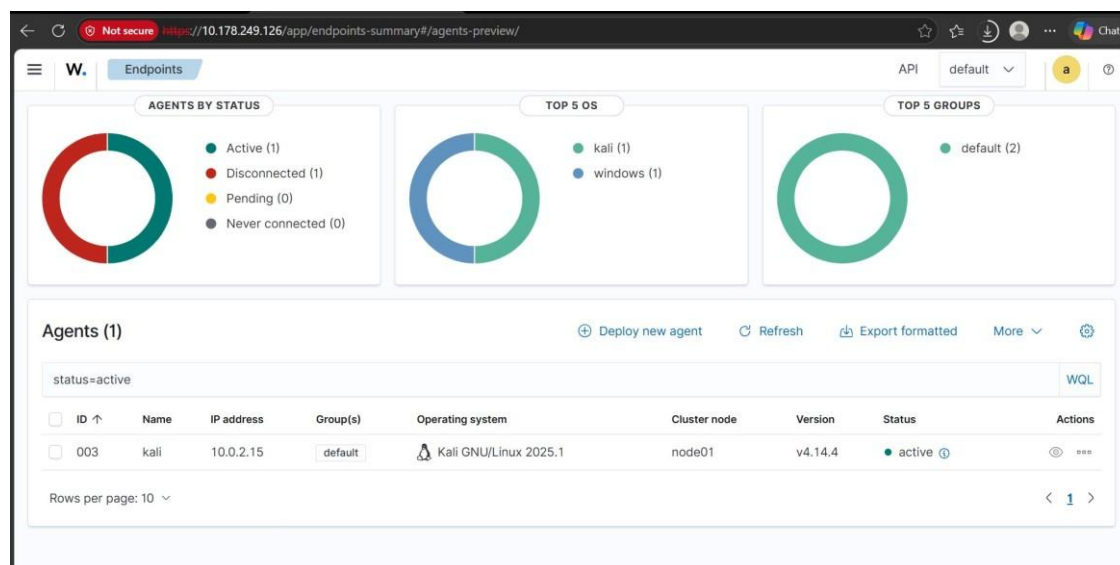


Figure 6: Wazuh Dashboard — Endpoints Module Showing Active Kali Agent (ID 003)

Figure 6 shows the Endpoints summary page. The Agents by Status chart confirms one active agent and one disconnected agent. The Top 5 OS chart shows the diversity of monitored endpoints: one Kali Linux agent and one Windows agent. The Agents table at the bottom provides specific details for the active agent: Agent ID 003, hostname "kali", IP address 10.0.2.15, operating system Kali GNU/Linux 2025.1, Wazuh version v4.14.4, and status active. This view confirms that the Kali agent was successfully registered and was actively communicating with the Wazuh Manager at the time of the screenshot.

10.3 Kali Agent Detail View

Clicking on the Kali agent in the Endpoints table opens the agent-specific detail page, which provides a comprehensive inventory of the endpoint and its security posture.

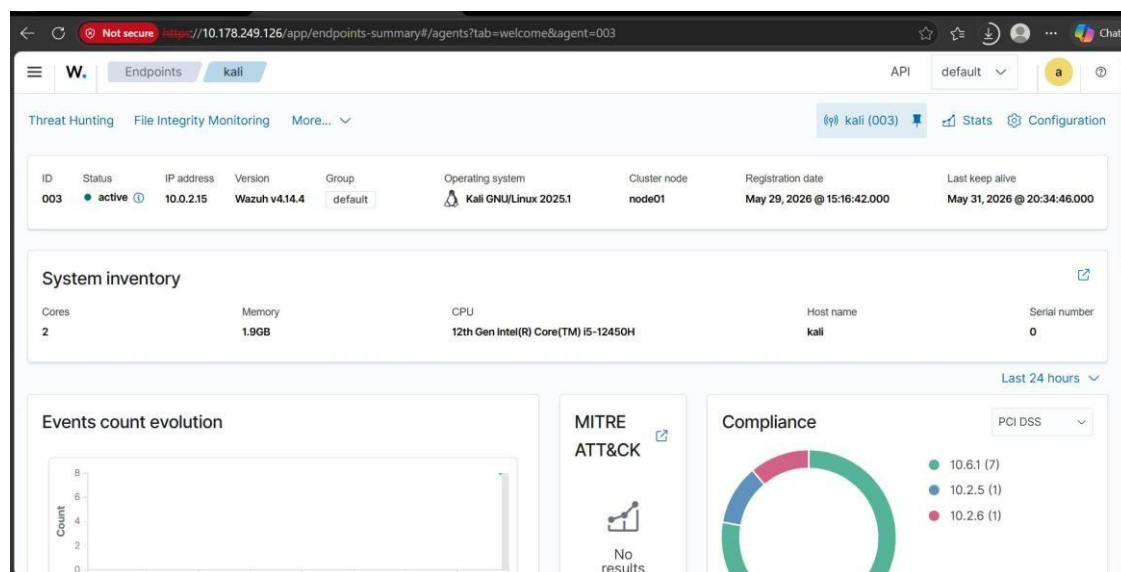


Figure 7: Wazuh Dashboard — Kali Agent (ID 003) Detail View with System Inventory

Figure 7 presents the detail view for Agent 003 (kali). The agent information panel at the top confirms the agent's status as active, registration date of 29 May 2026, and last keepalive timestamp of 31 May 2026 at 20:34:46. The System Inventory section reveals the hardware profile of the agent: 2 CPU cores, 1.9 GB of RAM, and a 12th Gen Intel Core i5-12450H processor. The hostname is confirmed as "kali". The lower portion of the page shows the Events count evolution chart (displaying recent event activity), the MITRE ATT&CK widget (indicating no MITRE-mapped results at the time of the screenshot), and the Compliance panel showing PCI DSS compliance breakdown with requirements 10.6.1 (7 events), 10.2.5 (1 event), and 10.2.6 (1 event).

10.4 Threat Hunting — Pre-Test Dashboard State

The Threat Hunting module provides a general security dashboard for investigating alerts associated with a specific agent.

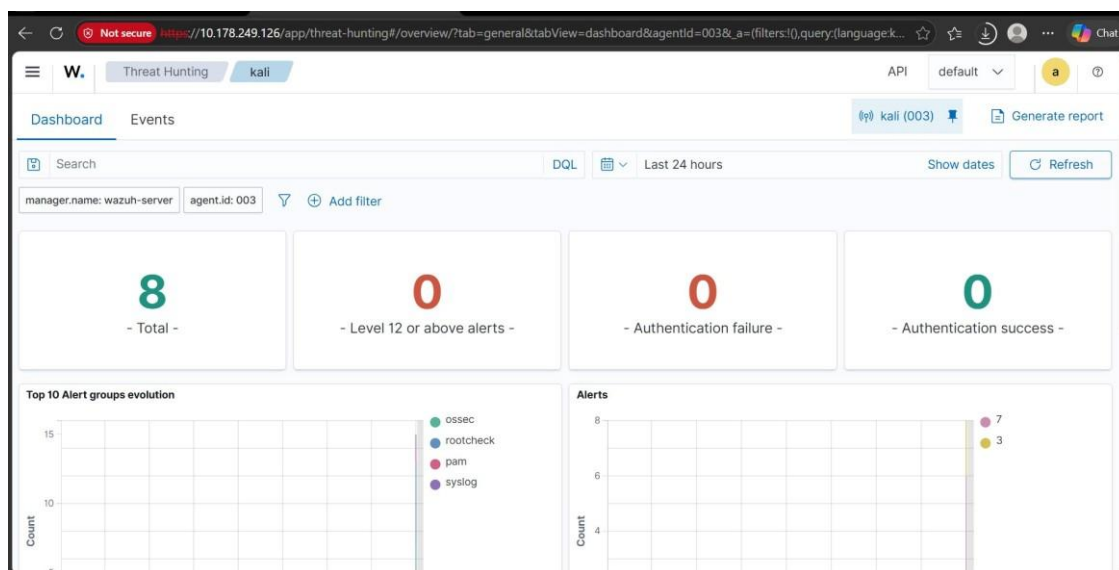


Figure 8: Wazuh Threat Hunting Dashboard — Pre-Test State (8 Total Events, 0 Critical)

Figure 8 shows the Threat Hunting dashboard for Agent 003 (kali) before the EICAR test was executed. At this point, 8 total events were recorded, with 0 Level 12 or above alerts, 0 authentication failures, and 0 authentication successes. The alert groups visible in the Top 10 Alert groups evolution chart include ossec, rootcheck, pam, and syslog — all standard system activity categories. This provides a baseline measurement against which the effect of the VirusTotal test can be assessed.

10.5 Threat Hunting — Post-Test Dashboard State

After the EICAR test file was introduced on the Kali agent, the Threat Hunting dashboard was refreshed to reflect the newly generated alerts.

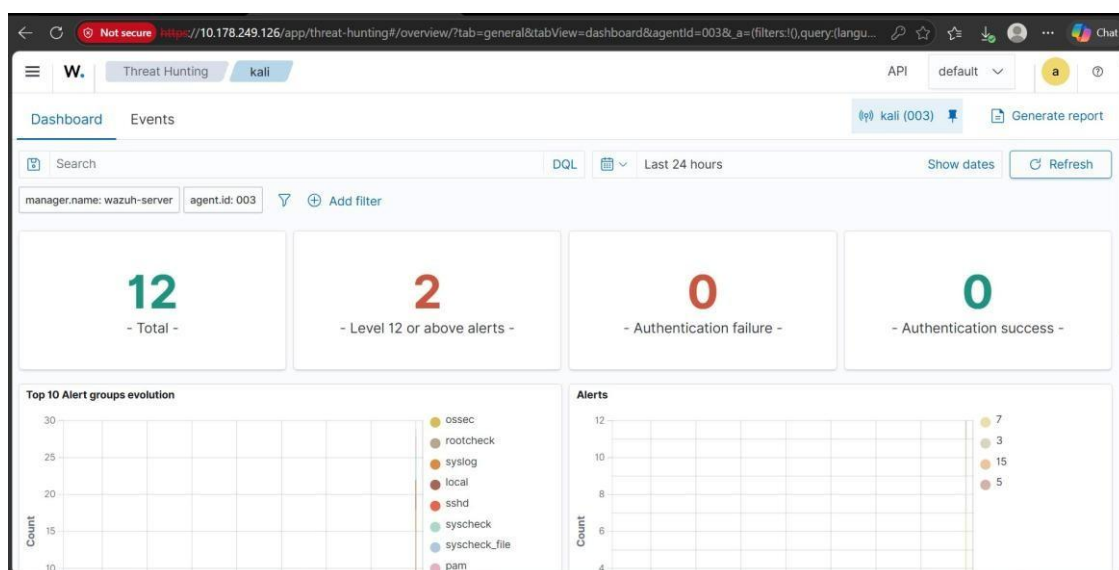
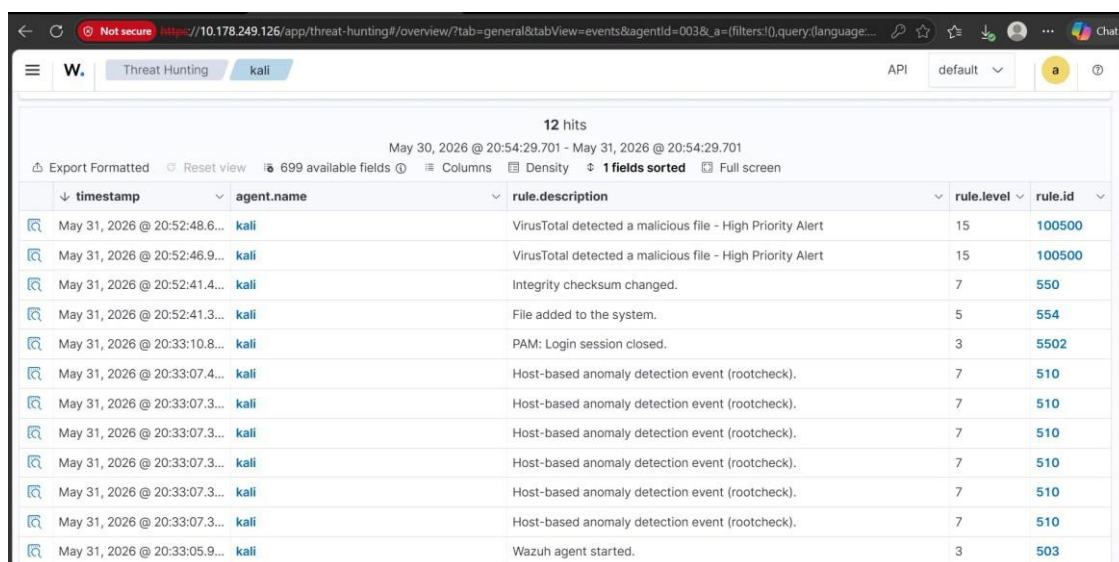


Figure 9: Wazuh Threat Hunting Dashboard — Post-Test State (12 Total Events, 2 Critical Level 12+ Alerts)

Figure 9 presents the Threat Hunting dashboard after the EICAR test. The total event count increased from 8 to 12, and critically, the Level 12 or above alerts counter increased from 0 to 2. This increase corresponds precisely to the two VirusTotal alerts generated by the EICAR file test (Rule ID 100500, Level 15). The appearance of syscheck and syscheck_file in the alert groups legend on the right confirms that the FIM module successfully detected the file creation event that triggered the VirusTotal integration. The Alerts chart on the right shows alert level 15 events (the custom VirusTotal rule), confirming that the highest-severity rule was triggered.

10.6 Threat Hunting Events — VirusTotal Alerts

The Events tab within the Threat Hunting module provides a tabular view of all individual security events, enabling analysts to inspect each alert in detail.



timestamp	agent.name	rule.description	rule.level	rule.id
May 31, 2026 @ 20:52:48.6...	kali	VirusTotal detected a malicious file - High Priority Alert	15	100500
May 31, 2026 @ 20:52:46.9...	kali	VirusTotal detected a malicious file - High Priority Alert	15	100500
May 31, 2026 @ 20:52:41.4...	kali	Integrity checksum changed.	7	550
May 31, 2026 @ 20:52:41.3...	kali	File added to the system.	5	554
May 31, 2026 @ 20:33:10.8...	kali	PAM: Login session closed.	3	5502
May 31, 2026 @ 20:33:07.4...	kali	Host-based anomaly detection event (rootcheck).	7	510
May 31, 2026 @ 20:33:07.3...	kali	Host-based anomaly detection event (rootcheck).	7	510
May 31, 2026 @ 20:33:07.3...	kali	Host-based anomaly detection event (rootcheck).	7	510
May 31, 2026 @ 20:33:07.3...	kali	Host-based anomaly detection event (rootcheck).	7	510
May 31, 2026 @ 20:33:07.3...	kali	Host-based anomaly detection event (rootcheck).	7	510
May 31, 2026 @ 20:33:07.3...	kali	Host-based anomaly detection event (rootcheck).	7	510
May 31, 2026 @ 20:33:05.9...	kali	Wazuh agent started.	3	503

Figure 10: Wazuh Threat Hunting Events — VirusTotal Malicious File Alerts (Rule ID 100500, Level 15)

Figure 10 presents the definitive evidence of the successful Wazuh-VirusTotal integration. The events table shows 12 total hits covering the period from 30 May 2026 to 31 May 2026. The top two events are of paramount significance:

- Timestamp: May 31, 2026 @ 20:52:48.6 — Agent: kali — Rule Description: VirusTotal detected a malicious file - High Priority Alert — Rule Level: 15 — Rule ID: 100500
- Timestamp: May 31, 2026 @ 20:52:46.9 — Agent: kali — Rule Description: VirusTotal detected a malicious file - High Priority Alert — Rule Level: 15 — Rule ID: 100500

These two entries confirm that Rule ID 100500 was triggered at Level 15 on the kali agent, precisely as configured. The rule description "VirusTotal detected a malicious file - High

Priority Alert" matches the description specified in the `local_rules.xml` file. The two alerts correspond to two separate VirusTotal API responses received for the EICAR file — one for the initial file creation detection and one for the integrity checksum change event.

The remaining events below the VirusTotal alerts include: Rule 550 (Integrity checksum changed, Level 7), Rule 554 (File added to the system, Level 5), Rule 5502 (PAM login session closed, Level 3), Rule 510 (Host-based anomaly detection, rootcheck, Level 7 — multiple entries), and Rule 503 (Wazuh agent started, Level 3). These events constitute normal system activity and the FIM events that directly preceded and triggered the VirusTotal integration.

11. Advantages and Limitations

11.1 Advantages

- **Cost Effectiveness:** Wazuh is open-source and freely available, and the VirusTotal free tier provides sufficient capacity for small-scale deployments. This makes enterprise-grade malware detection accessible to organisations with limited budgets.
- **Real-Time Detection:** The integration delivers near-real-time malware detection. From the moment a file is created on the monitored endpoint to the appearance of a Level 15 alert in the Dashboard, the latency is typically under 60 seconds, constrained primarily by the VirusTotal API polling interval.
- **Crowdsourced Intelligence:** VirusTotal aggregates results from over 70 antivirus engines. A file flagged by even a subset of these engines triggers an alert, providing a significantly broader detection capability than any single antivirus product.
- **Extensibility:** The Wazuh integration framework is not limited to VirusTotal. The same architectural pattern can be applied to integrate other threat intelligence services, SOAR platforms, and ticketing systems.
- **MITRE ATT&CK Mapping:** The custom rule includes a MITRE ATT&CK technique ID (T1204), enabling automated mapping of alerts to the adversary behaviour framework. This enriches the alert data and supports threat hunting activities.
- **Centralised Visibility:** All alerts from multiple agents are aggregated in the Wazuh Dashboard, providing a unified view of the security posture across the monitored environment.

11.2 Limitations

- **API Rate Limiting:** The free VirusTotal API tier is restricted to 4 requests per minute and 500 requests per day. In environments with high file creation rates, this constraint may result in missed detections or delayed alerting. Commercial API tiers mitigate this limitation.
- **Hash-Based Detection Only:** The current integration queries VirusTotal by file hash. Files that have not been previously submitted to VirusTotal will not be detected, as VirusTotal cannot analyse files it has not indexed. This limitation affects zero-day malware.
- **No Automated Remediation:** The current implementation generates alerts but does not automatically quarantine or delete malicious files. Manual analyst intervention is required to respond to alerts, which introduces a time lag between detection and remediation.
- **Network Dependency:** The integration requires outbound internet connectivity from the Wazuh Manager to the VirusTotal API. In air-gapped or highly restricted network environments, this connectivity may not be available.
- **False Positives:** Legitimate files that share a hash with known malware samples (a theoretical but non-zero possibility) would generate false positive alerts.

Additionally, legitimate software that happens to share structural characteristics with known malware may be flagged by some VirusTotal engines.

-
- **Single Point of Failure:** In the current laboratory configuration, the Wazuh Manager is a single node. Production deployments should implement clustering and high availability to ensure continuous monitoring.

12. Conclusion

This project successfully demonstrated the design, implementation, and validation of an automated malware detection system integrating the Wazuh open-source SIEM platform with the VirusTotal threat intelligence API. The system was deployed in a controlled virtualised environment comprising a Wazuh Manager (Ubuntu 22.04 LTS) and a monitored Kali GNU/Linux 2025.1 endpoint, mirroring real-world SOC deployment scenarios.

The implementation confirmed that the Wazuh File Integrity Monitoring module, when combined with the VirusTotal integration and appropriately configured custom detection rules, delivers a comprehensive and effective malware detection pipeline. The EICAR test file validation produced two Level 15 (highest severity) alerts in the Wazuh Dashboard within seconds of the malicious file being created on the endpoint, demonstrating the practical effectiveness of the solution.

From a technical standpoint, the project provided deep practical experience with the configuration of a production-grade SIEM platform, including the authoring of custom XML detection rules, the integration of a REST API threat intelligence service, the navigation and interpretation of the Wazuh Dashboard, and the troubleshooting of agent connectivity and configuration issues encountered during deployment.

From a professional development perspective, the project provided valuable exposure to the workflows and tooling employed by SOC analysts in real-world security operations. The ability to demonstrate a working Wazuh-VirusTotal integration pipeline, explain its architecture, and interpret its alert data constitutes a meaningful credential for roles in cybersecurity operations, incident response, and threat analysis.

Future enhancements to this implementation could include the activation of automated active responses to quarantine or terminate processes associated with detected malicious files, the integration of additional threat intelligence feeds (such as AbuseIPDB or OTX AlienVault) to extend detection coverage to network indicators of compromise, and the deployment of a Wazuh cluster with multiple manager nodes to eliminate the single point of failure present in the current configuration. The integration of Wazuh with a SOAR (Security Orchestration, Automation, and Response) platform such as TheHive

or Cortex would further automate the incident response workflow, reducing mean time to response (MTTR) and minimising analyst workload.

In conclusion, this project demonstrates that open-source security platforms, when properly configured and integrated with external threat intelligence services, can deliver detection capabilities comparable to commercial solutions at a fraction of the cost. This finding is particularly relevant for small and medium-sized organisations seeking to establish or mature their security operations capabilities without incurring substantial licensing expenditure.

13. References

- [1] Wazuh, Inc. (2024). Wazuh Documentation — VirusTotal Integration. Retrieved from <https://documentation.wazuh.com/current/user-manual/capabilities/virustotalscan/index.html>
- [2] Wazuh, Inc. (2024). Wazuh Documentation — File Integrity Monitoring. Retrieved from <https://documentation.wazuh.com/current/user-manual/capabilities/fileintegrity/index.html>
- [3] Wazuh, Inc. (2024). Wazuh Documentation — Custom Rules and Decoders. Retrieved from <https://documentation.wazuh.com/current/user-manual/ruleset/custom.html>
- [4] Google LLC. (2024). VirusTotal API v3 Documentation. Retrieved from <https://developers.virustotal.com/reference/overview>
- [5] MITRE Corporation. (2024). MITRE ATT&CK Framework — T1204: User Execution. Retrieved from <https://attack.mitre.org/techniques/T1204/>
- [6] EICAR — European Institute for Computer Antivirus Research. (2024). EICAR Standard Anti-Virus Test File. Retrieved from <https://www.eicar.org/download-antimalware-testfile/>
- [7] OpenSearch Project. (2024). OpenSearch Dashboards Documentation. Retrieved from <https://opensearch.org/docs/latest/dashboards/index/>
- [8] Ubuntu. (2024). Ubuntu 22.04 LTS (Jammy Jellyfish) Release Notes. Retrieved from <https://releases.ubuntu.com/22.04/>
- [9] Kali Linux. (2025). Kali Linux 2025.1 Release. Retrieved from <https://www.kali.org/blog/kali-linux-2025-1-release/>
- [10] Oracle Corporation. (2024). Oracle VM VirtualBox User Manual. Retrieved from <https://www.virtualbox.org/manual/>